Kiến trúc SOA & Microservices

Đặng Ngọc Hùng

Mục lục

9. Bảo mật Microservices — OAuth 2.0, mTLS & Zero Trust . . . 2
9.1. Các thách thức bảo mật trong kiến trúc microservices . . . 3
9.1.1. Vấn đề: bề mặt tấn công (attack surface) mở rộng . . . 3
9.1.2. Nguyên tắc phòng thủ chiều sâu . . . 4
9.1.3. mTLS, Secret Management và OAuth2 scope . . . 5
9.2. JWT — cấu trúc và cơ chế . . . 6
9.2.1. Vấn đề: giới hạn khả năng mở rộng của phiên làm việc (session) trong microservices . . . 6
9.2.2. JWT (JSON Web Token) . . . 6
9.2.3. Luồng JWT mục tiêu và KBM tại thời điểm khảo sát . . . 7
9.3. Tiered Trust và xác minh theo tài nguyên . . . 11
9.3.1. Vấn đề: ba phép kiểm tra thường bị gộp làm một . . . 11
9.3.2. KBM tại thời điểm khảo sát và mục tiêu . . . 12
9.4. OAuth2 — nhà cung cấp danh tính bên ngoài . . . 13
9.4.1. OAuth2 Authorization Code Flow . . . 13
9.4.2. Nhiều phương thức xác thực . . . 14
9.4.3. Service-to-service: Client Credentials Flow . . . 14
9.4.4. Không có trình duyệt: Device Authorization Grant . . . 14
9.5. RBAC — Role-Based Access Control . . . 15
9.5.1. Vấn đề: ai được làm gì? . . . 15
9.5.2. Triển khai RBAC trong KBM . . . 15
9.5.3. Điều khiển hiển thị route từ API quyền (frontend) . . . 16
9.6. Case study: KBM . . . 17
9.6.1. Phân tích tổng hợp . . . 17
9.6.2. Lộ trình cải tiến . . . 18
9.6.3. Secret Management trong production . . . 19
9.6.4. Cô lập thực thi và phong tỏa (lockdown) thi cử — hai bài học phòng thủ . . . 21
9.7. Tổng kết . . . 22
9.8. Đọc thêm . . . 23
Tài liệu tham khảo . . . 24

CHƯƠNG 9

Bảo mật Microservices — OAuth 2.0, mTLS & Zero Trust

> "Some organizations attempt to ensure security at the perimeter of their networks, and therefore assume they don't need to do anything else when two services are talking together. However, should an attacker penetrate your network, you will have little protection against a typical man-in-the-middle attack."
> — Sam Newman, *Building Microservices* [4a, Ch.9]

MỤC TIÊU HỌC TẬP

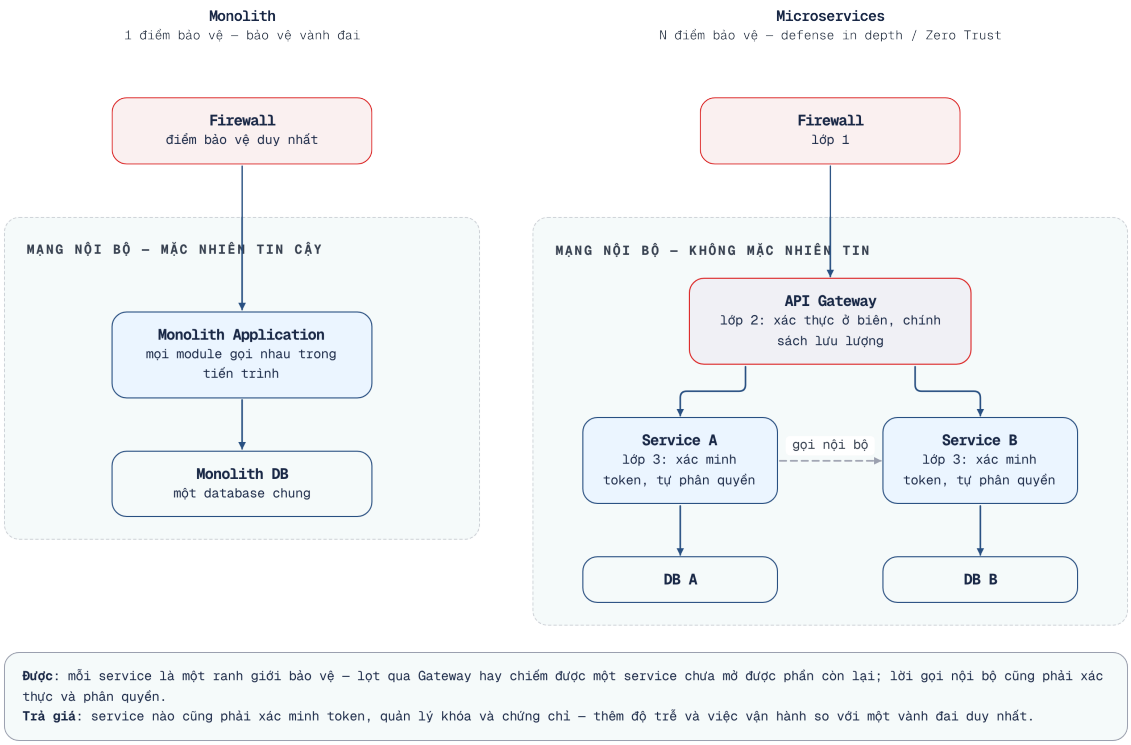
- Hiểu các thách thức bảo mật đặc thù của kiến trúc microservices
- Nắm vững cấu trúc JWT (JSON Web Token) dạng compact JWS: header, payload, signature — và phân biệt với dạng JWE năm phần khi cần mã hóa
- Phân biệt kiểm tra ở biên, xác minh tại service đích và đối chiếu trạng thái tại cơ sở dữ liệu; áp dụng tầng tin cậy (trust tier) theo mức rủi ro
- Hiểu OAuth2/OIDC (OpenID Connect) flow và tích hợp nhà cung cấp danh tính bên ngoài (Microsoft/Google)
- Thiết kế RBAC (Role-Based Access Control) cho hệ thống đa vai trò
- Tách biệt user subject, calling workload/actor và delegation trong giao tiếp service-to-service
- Phân tích kiến trúc bảo mật của KBM

9.1. Các thách thức bảo mật trong kiến trúc microservices

Việc bảo mật một hệ thống nguyên khối giống như bảo vệ một pháo đài duy nhất, nhưng với microservices, bài toán này biến thành việc phải bảo vệ từng tòa nhà, từng giảng đường nằm rải rác khắp khuôn viên đại học.

9.1.1. Vấn đề: bề mặt tấn công (attack surface) mở rộng

Trong kiến trúc nguyên khối (monolith), bảo mật tập trung tại một điểm: một ứng dụng, một cơ sở dữ liệu, một tầng xác thực (authentication layer). Khi chuyển sang microservices, bề mặt tấn công mở rộng tỷ lệ thuận với số lượng dịch vụ:



Hình 9.1: *Monolith (1 điểm bảo vệ) vs Microservices (N điểm bảo vệ)*

Tổng hợp từ *Building Microservices* của Sam Newman, có thể nêu năm thách thức bảo mật đặc thù khi chuyển dịch sang hệ thống phân tán, nơi sự tin tưởng mặc định không còn tồn tại và mỗi thành phần đều phải tự thiết lập cơ chế bảo vệ:

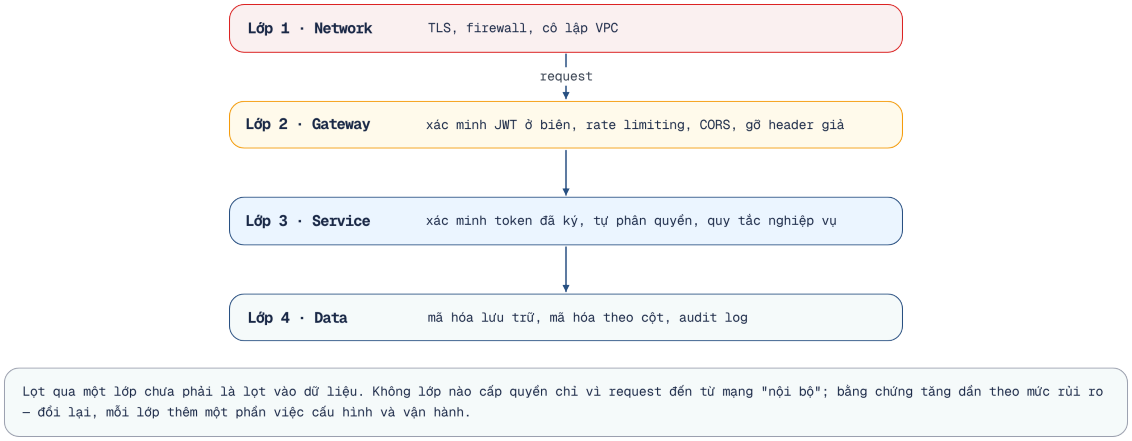
Bảng 9.1: *Năm thách thức bảo mật đặc thù khi chuyển từ monolith sang microservices*

#	Thách thức	Monolith	Microservices
1	**Xác thực**	Một session store	Gateway kiểm tra credential ở biên; resource service xác minh bằng chứng (proof) phù hợp với trust tier
2	**Phân quyền**	Kiểm tra cục bộ (in-process)	Mỗi dịch vụ cần phải kiểm tra quyền truy cập — người dùng được phép thực hiện những hành động nào?
3	**Tin cậy service-to-service**	Không có (in-process)	Dịch vụ B phải phân biệt workload A đang gọi, user subject (nếu có) và quyền delegation
4	**Dữ liệu trên đường truyền**	Nội bộ (in-process)	Lời gọi mạng — cần mã hóa (TLS)

#	Thách thức	Monolith	Microservices
5	**Secret Management**	Một file cấu hình	N dịch vụ kết hợp với M thông tin bí mật (secret) = bài toán quản lý phức tạp

9.1.2. Nguyên tắc phòng thủ chiều sâu

Để đối phó với các thách thức trên, bảo mật trong môi trường phân tán phải dựa trên nguyên tắc **phòng thủ chiều sâu** (defense in depth). Thay vì chỉ phụ thuộc vào một tường lửa duy nhất ở biên mạng, chúng ta phải xây dựng nhiều rào chắn bảo vệ nối tiếp nhau từ tầng mạng cho đến tận tầng dữ liệu:



Hình 9.2: *Phòng thủ chiều sâu — bảo vệ nhiều lớp từ tầng mạng đến tầng dữ liệu*

Trong thực tế, mức độ bảo vệ tùy thuộc vào **ranh giới tin cậy (trust boundary)**, nhưng vị trí trong private network/VPC không tự biến request thành đáng tin cậy. Gateway tạo một ranh giới vào (ingress) quan trọng; mỗi service đích vẫn là một ranh giới tài nguyên (resource boundary) vì service có dữ liệu và quy tắc phân quyền riêng.

Tin cậy phân tầng (Tiered Trust), không chỉ tin cậy ở biên

Zero Trust không có nghĩa là lặp lại truy vấn database ở mọi hop; nó yêu cầu không cấp quyền chỉ vì request đến từ một vị trí mạng được xem là "nội bộ". Thiết kế thực dụng là tăng bằng chứng theo mức rủi ro: làm sạch request ở biên (ingress sanitation), bằng chứng có chữ ký (signed proof) đúng audience, phân quyền cục bộ (local authorization), rồi bổ sung workload identity hoặc delegation khi cần.

Sách sử dụng bốn tầng tin cậy (trust tier) thống nhất. **Tier 0** là request ngoài chưa được tin cậy: Gateway phải làm sạch header và xác minh token. **Tier 1** là luồng nội bộ thông thường: resource service xác minh bằng chứng danh tính có chữ ký và tự phân quyền. **Tier 2** áp dụng cho nghiệp vụ nhạy cảm: ngoài user subject còn phải nhận diện calling workload/actor và delegation. **Tier 3** dành cho job hoặc command bất đồng bộ: consumer xác minh workload cùng quyền thực thi của job/command, không giả định rằng message broker đã tạo ra sự tin cậy.

9.1.3. mTLS, Secret Management và OAuth2 scope

Phần nâng cao (có thể lướt qua)

Ba chủ đề dưới đây (*mTLS*, *Secret Management*, *OAuth2 scope*) mở rộng cho các hệ thống có mức rủi ro cao hoặc tích hợp đối tác bên ngoài. Trong lần đọc đầu, người đọc có thể đọc lướt để nắm khái niệm và quay lại đối chiếu khi xem phần phân tích KBM ở §9.6.

Khi hệ thống mở rộng quy mô hoặc mức độ rủi ro tăng lên (ví dụ: bổ sung phân hệ thanh toán, xử lý dữ liệu cá nhân), cần nâng cấp các biện pháp bảo mật:

mTLS (Mutual TLS) — Trong TLS thông thường, chỉ có máy khách xác minh máy chủ (ví dụ trình duyệt xác minh chứng chỉ HTTPS). Trong mTLS, **cả hai bên đều xác minh chứng chỉ lẫn nhau**: khi dịch vụ A gọi dịch vụ B, B kiểm tra chứng chỉ của A và A kiểm tra chứng chỉ của B. Cơ chế này bảo vệ kênh truyền và xác thực workload ở hai đầu kết nối.

Khác với giao tiếp nội bộ qua HTTP không mã hóa, mTLS giảm nguy cơ nghe lén và giả mạo workload. Tuy nhiên, chứng chỉ của service A không chứng minh rằng claim `userId` do A chuyển tiếp là đúng; user subject vẫn cần signed proof hoặc delegation có thể xác minh. Cái giá phải đánh đổi là độ phức tạp khi cấp phát, luân phiên và thu hồi chứng chỉ; việc này thường cần công cụ như cert-manager hoặc service mesh.

Tại thời điểm khảo sát (đầu năm 2026), mTLS chưa được xác nhận là đã triển khai trong KBM. Đây là khoảng trống được chấp nhận tạm thời của một hệ thống học thuật, không phải lý do để coi mạng nội bộ là bằng chứng danh tính. Khi mức độ rủi ro tăng, mTLS cùng NetworkPolicy giúp thu hẹp khả năng di chuyển ngang (lateral movement); bằng chứng danh tính người dùng có chữ ký và phân quyền cục bộ vẫn là các lớp độc lập.

Secret Management — Các thông tin bí mật (secret như mật khẩu cơ sở dữ liệu, khóa API, khóa bí mật JWT) không nên được lưu trữ trực tiếp trong mã nguồn hoặc image Docker.

Trong việc quản lý thông tin nhạy cảm, gắn cố định (hardcode) secret trực tiếp vào mã nguồn (như `password: "abc123"`) là mức rủi ro cao nhất: một khi đã commit, secret nằm lại vĩnh viễn trong lịch sử Git. Cấp độ bảo mật trung bình là sử dụng biến môi trường (environment variable) qua file `.env`: phương án này tách bạch mã nguồn khỏi cấu hình, nhưng secret vẫn dễ bị đánh cắp nếu kẻ tấn công rà quét tiến trình chạy container. Giải pháp tiêu chuẩn cho hệ thống production là dùng công cụ quản lý secret chuyên dụng (Secret Manager) như HashiCorp Vault, cung cấp cơ chế mã hóa khi lưu trữ (encryption at rest), tự động luân phiên khóa và kiểm toán truy cập.

Tại thời điểm khảo sát, KBM dùng biến môi trường cho một phần cấu hình, còn một phần secret vẫn gắn cố định trong tệp cấu hình (§9.6). Dù ở hệ thống nào, khóa ký JWT cũng cần nằm ngoài mã nguồn và được luân phiên (rotation) định kỳ.

OAuth2 scope — Hiện tại RBAC của KBM dùng các vai trò (`STUDENT`, `LECTURER`, `ADMIN`). OAuth2 scope mở rộng thêm chiều kiểm soát: thay vì "người dùng có vai trò `ADMIN`", phạm vi quyền (scope) cho phép chỉ định "ứng dụng máy khách X có quyền đọc bài nộp nhưng không có quyền ghi điểm". Cơ chế này đặc biệt phù hợp khi KBM cung cấp API cho các bên thứ ba (như ứng dụng di động độc lập hoặc tích hợp với các hệ thống khác).

9.2. JWT — cấu trúc và cơ chế

9.2.1. Vấn đề: giới hạn khả năng mở rộng của phiên làm việc (session) trong microservices

Trong kiến trúc nguyên khối, quá trình xác thực thường sử dụng cơ chế phiên làm việc phía máy chủ (**server-side session**): người dùng đăng nhập, máy chủ tạo một phiên làm việc, lưu trữ trong bộ nhớ hoặc Redis, sau đó trả về mã định danh phiên (session ID) dưới dạng cookie. Các yêu cầu tiếp theo sẽ gửi kèm cookie này để máy chủ tra cứu và nhận diện người dùng.

Trong kiến trúc microservices, kho lưu trữ phiên (session store) tạo ra một điểm phụ thuộc tập trung (**centralized dependency**): việc mọi dịch vụ phải truy vấn cùng một cơ sở dữ liệu phiên sẽ gây ra điểm nghẽn và tạo thành điểm lỗi duy nhất (single point of failure). Ngược lại, nếu mỗi dịch vụ có một kho lưu trữ phiên riêng, người dùng sẽ phải đăng nhập lại mỗi khi chuyển đổi giữa các dịch vụ.

9.2.2. JWT (JSON Web Token)

Thi giấy vs thi máy tính

- **Session (Thi giấy)**: Giám thị phải mang theo một danh sách điểm danh. Mỗi khi có sinh viên nộp bài, giám thị phải tra đúng dòng, đúng tên trong danh sách (tra cứu session store). Nếu giám thị chuyển danh sách đi phòng khác, hệ thống sẽ rối loạn.
- **JWT (Thi máy tính)**: Giống như một **mã QR định danh** do hệ thống của Phòng Khảo thí cấp khi đăng nhập. Sinh viên cầm mã QR này nộp bài. Hệ thống chấm bài (service) chỉ cần quét xem mã QR có chữ ký điện tử hợp lệ của Phòng Khảo thí hay không và đọc (decode) được mã sinh viên mà không cần truy vấn cơ sở dữ liệu.

JWT giải quyết vấn đề này bằng cách đóng gói thông tin định danh *vào chính token* kèm chữ ký số — stateless, không cần kho lưu trữ phiên [2a, Ch.11]:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.            ← Header
eyJ1c2VySWQiOiJ1c2VyLTEyMyIsInJvbGVzIjoiU1R      ← Payload
VREVOVCIsImV4cCI6MTg5MzQ1NjAwMH0.
SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c      ← Signature
```

Ba phần của một JWT dạng compact JWS:



Hình 9.3: *Cấu trúc JWT dạng compact JWS — Header, Payload, Signature*

Về mặt cấu trúc, JWT minh họa ở đây là *compact JWS* — dạng phổ biến nhất — gồm ba phần được encode Base64URL riêng biệt (JWT dạng JWE mã hóa có năm phần). Phần đầu tiên là **Header**

chứa thông tin về thuật toán ký và loại token (ví dụ: `{"alg": "HS256"}`). Phần thứ hai là **Payload**, mang theo các dữ liệu định danh (claim) quan trọng của người dùng như `userId` hay `roles`. Thành phần cuối cùng là **Signature**, bảo vệ tính toàn vẹn của token bằng cách ký lên Header và Payload — HS256 dùng HMAC với khóa bí mật, RS256 ký bằng private key (bảng so sánh bên dưới) — giúp ngăn chặn việc chỉnh sửa dữ liệu trái phép.

Cần lưu ý rằng Base64URL chỉ là *encoding*, không phải mã hóa: bất kỳ ai cầm token đều đọc được toàn bộ payload. Vì vậy tuyệt đối không đặt mật khẩu hay dữ liệu nhạy cảm trong claim; khi thật sự cần giữ bí mật nội dung, hãy dùng JWE (JSON Web Encryption), lược bỏ claim nhạy cảm khỏi payload, hoặc chuyển sang opaque token — tùy mô hình mối đe dọa (threat model).

So sánh HS256 và RS256: Hai thuật toán phổ biến này khác nhau chủ yếu ở cách ký và cách xác minh chữ ký:

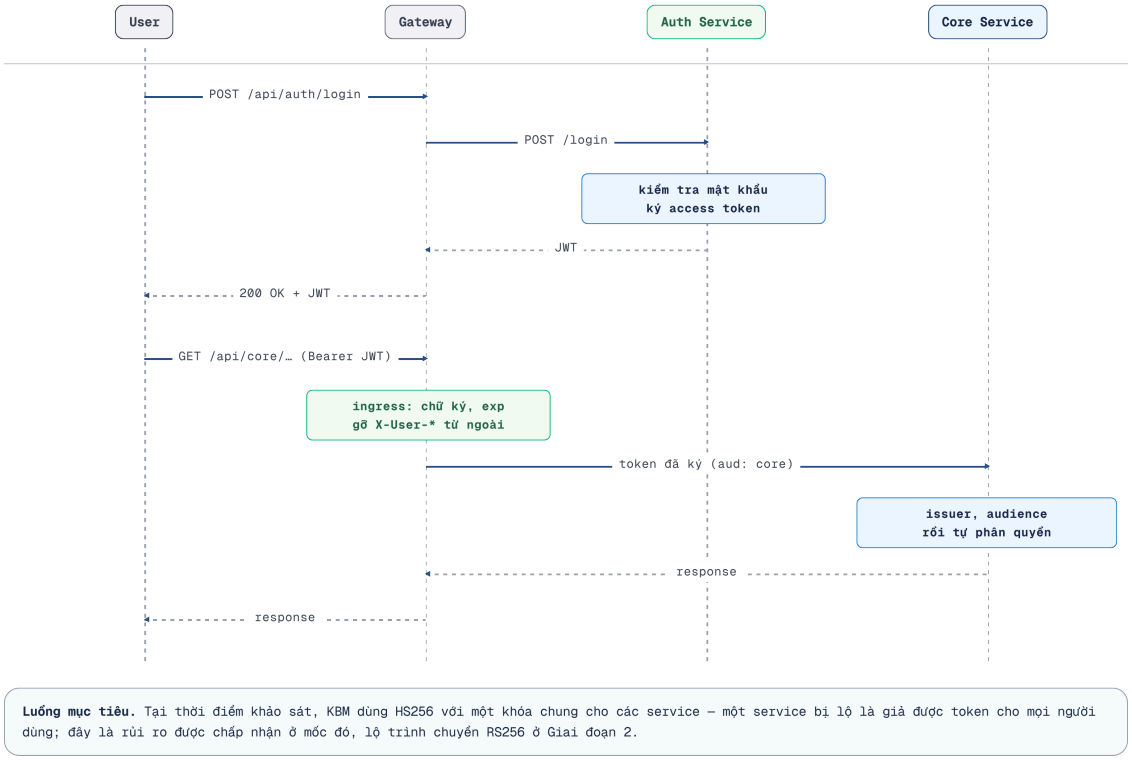
Bảng 9.2: *HS256 (đối xứng) vs RS256 (bất đối xứng)*

	HS256 (đối xứng)	RS256 (bất đối xứng)
Khóa	Khóa bí mật dùng chung (shared secret key)	Khóa riêng tư (ký) + Khóa công khai (xác minh)
Ai ký?	Auth Service (có khóa bí mật)	Auth Service (có khóa riêng tư)
Ai xác minh?	Bất kỳ dịch vụ nào có khóa bí mật	Bất kỳ dịch vụ nào có khóa công khai
Bảo mật	Nếu khóa bí mật (secret key) bị lộ, cả quá trình ký và xác minh đều bị xâm phạm	Khóa công khai phân phối rộng rãi không gây rủi ro; nhưng nếu khóa riêng tư (private key) bị lộ, kẻ tấn công ký được token giả mà mọi dịch vụ vẫn xác minh "hợp lệ" — phải thu hồi và luân phiên khóa khẩn cấp
Luân phiên khóa	Phải phân phối an toàn secret mới cho mọi bên xác minh (verifier)	Auth Service công bố khóa công khai mới qua JWKS; verifier tự làm mới cache
Phù hợp khi	Phạm vi hẹp, ít verifier và kiểm soát secret chặt	Nhiều resource service/verifier; chỉ issuer giữ private key

Dù dùng HS256 hay RS256, resource service không được suy ra tính hợp lệ chỉ từ việc decode thành công. Hợp đồng xác minh tối thiểu phải cố định thuật toán được phép, kiểm tra chữ ký, đối chiếu chính xác issuer và audience, rồi kiểm tra `exp` cùng `nbf` trước khi dùng claim để phân quyền. Token dành cho Gateway không mặc nhiên hợp lệ ở mọi dịch vụ tuyến sau (downstream service); nếu audience khác nhau, cần token exchange hoặc internal token riêng.

9.2.3. Luồng JWT mục tiêu và KBM tại thời điểm khảo sát

Hình 9.4 là luồng JWT mục tiêu: Gateway xác minh token ở biên, service đích nhận bằng chứng có chữ ký và tự phân quyền cục bộ. KBM tại thời điểm khảo sát còn cách mục tiêu này một khoảng; phần phân tích nằm ngay sau hình.



Hình 9.4: Luồng JWT mục tiêu — kiểm tra ở ingress, bằng chứng có chữ ký và phân quyền cục bộ

HS256 với shared secret trong KBM

Tại thời điểm khảo sát, KBM dùng HS256 (đối xứng) — các dịch vụ chia sẻ cùng một khóa bí mật. Đây là rủi ro đáng kể: nếu bất kỳ dịch vụ nào bị xâm nhập, kẻ tấn công có thể tạo lập chuỗi JWT giả mạo cho **bất kỳ người dùng nào trên toàn hệ thống** (không giới hạn ở dịch vụ bị tấn công). Với KBM học thuật, đây là **rủi ro được chấp nhận (accepted risk)** ở mốc đó, được ghi rõ, không phải mô hình mục tiêu cho production. **Lộ trình chuyển đổi** được xếp vào Giai đoạn 2 vì cần hạ tầng quản lý khóa: (1) Auth Service chuyển sang RS256 và là nơi duy nhất giữ private key; (2) các resource service dùng Spring Security OAuth2 Resource Server để xác minh theo cùng issuer/audience; (3) khóa công khai được luân phiên qua JWKS; (4) Gateway phát hành hoặc trao đổi sang internal token đúng audience khi không thể chuyển tiếp external token nguyên trạng.

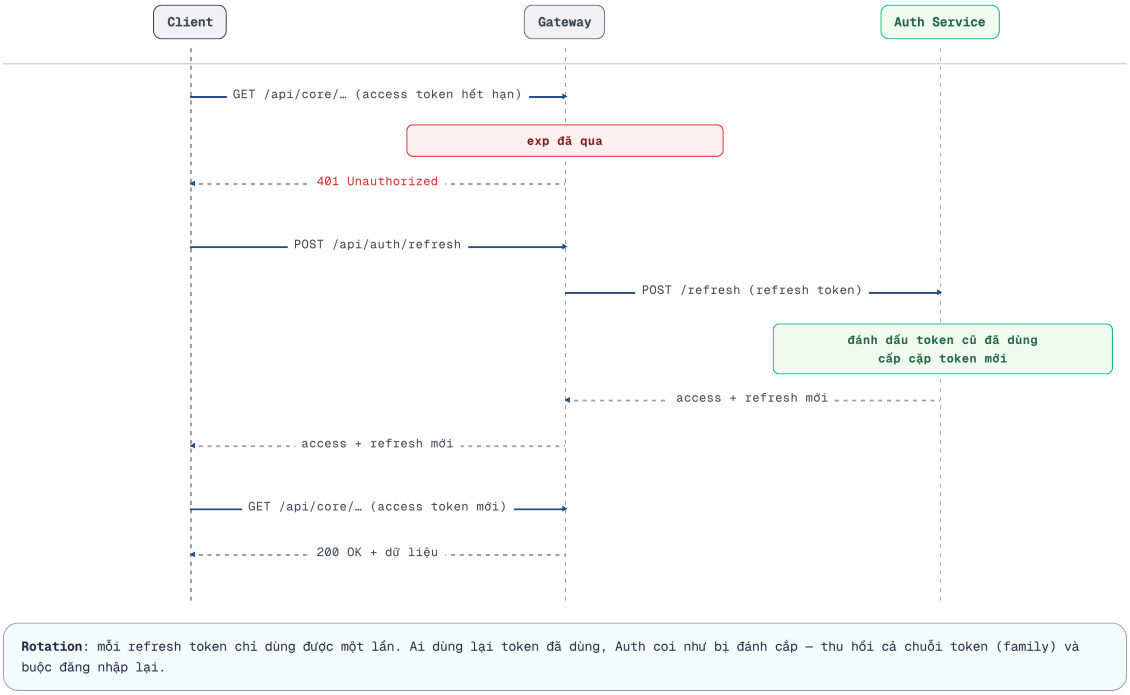
Việc luân phiên khóa (key rotation) trong thực tế vướng một rào cản kỹ thuật cụ thể.

Cửa sổ dual-secret khi luân phiên khóa HS256

Luân phiên khóa bí mật của HS256 không thể thực hiện tức thời: nếu Auth Service chuyển sang khóa bí mật mới trong khi các dịch vụ khác vẫn giữ khóa bí mật cũ, các token mới ký bằng khóa mới sẽ bị các dịch vụ đó từ chối; còn nếu gỡ khóa cũ quá sớm, các token cũ còn hạn sẽ bị từ chối. Cách an toàn là duy trì một **cửa sổ dual-secret**: trong giai đoạn chuyển tiếp, khâu *xác minh* của các dịch vụ sẽ chấp nhận **cả khóa cũ lẫn khóa mới** (`current` + `previous`), còn Auth Service sẽ *ký* bằng khóa mới; chỉ gỡ bỏ khóa cũ sau khi mọi token được ký bằng nó đã hoàn toàn hết hạn. Sự phức tạp này là một lý do nữa để chuyển sang cơ chế RS256/JWKS — khi đó chỉ cần công bố

thêm khóa công khai mới qua endpoint JWKS, không cần phải đồng bộ khóa bí mật qua tất cả các dịch vụ.

JWT có thời hạn (`exp`). Khi access token hết hạn, người dùng phải **xác thực lại (re-authenticate)** — điều này làm trải nghiệm kém đi ở các phiên làm việc kéo dài (ví dụ: giảng viên sử dụng hệ thống liên tục suốt một buổi giảng). **Refresh token** giải quyết bài toán này: access token có thời hạn ngắn (15–60 phút), còn refresh token có thời hạn dài hơn (7–30 ngày). Khi access token hết hạn, client dùng refresh token để nhận access token mới — không cần đăng nhập lại.



Hình 9.5: Luồng làm mới và luân phiên token — access token mới + refresh token mới

Token rotation là chiến lược luân phiên refresh token: mỗi lần refresh token được sử dụng, **bản thân nó cũng được thay mới** và bản cũ bị vô hiệu hóa. Lợi ích của cơ chế này là: nếu refresh token bị đánh cắp và dùng lại trái phép, Auth Service sẽ phát hiện token đó đã qua sử dụng. Khi đó, hệ thống sẽ tự động vô hiệu hóa toàn bộ chuỗi token liên quan và buộc người dùng phải đăng nhập lại từ đầu (RFC 9700).

Đăng ký vé dùng máy tính thư viện

- **Access token**: tương tự *vé sử dụng máy tính thư viện 1 giờ* (ngắn hạn để nếu lộ thông tin vé, kẻ gian chỉ dùng được tối đa trong 1 giờ).
- **Refresh token**: tương tự *thẻ sinh viên gốc*, dùng để xuất trình cho thủ thư cấp vé 1 giờ mới khi hết hạn.
- **Token rotation**: mỗi lần xin vé mới, thủ thư dán *tem gia hạn mới* lên thẻ và hủy hiệu lực tem cũ — refresh token vì thế đổi mới sau mỗi lần dùng. Nếu kẻ gian mang tem cũ đã hủy ra dùng lại, hệ thống phát hiện ngay và **thu hồi toàn bộ chuỗi tem** của thẻ đó.

Listing 9.1: *Endpoint làm mới token có luân phiên*

```
// Auth Service — Token refresh với rotation
// @Transactional: đánh dấu token cũ đã dùng + phát hành token mới
// trong CÙNG transaction.
// noRollbackFor: lệnh thu hồi family phải được COMMIT ngay cả khi
// chúng ta chủ động ném TokenReuseException để từ chối request
// (mặc định Spring rollback theo RuntimeException)
@Transactional(noRollbackFor = TokenReuseException.class)
@PostMapping("/refresh")
public TokenResponse refreshToken(@RequestBody RefreshRequest request) {
    RefreshToken stored = refreshTokenRepository
        .findByToken(request.getRefreshToken())
        .orElseThrow(() -> new AuthException("Invalid refresh token"));

    // Kiểm tra thu hồi và dùng lại TRƯỚC hạn dùng: token cũ đã hết hạn
    // mà bị dùng lại vẫn phải kích hoạt thu hồi cả chuỗi (RFC 9700)
    if (stored.isRevoked()) {
        throw new AuthException("Refresh token revoked");
    }
    if (stored.isUsed()) {
        refreshTokenRepository.invalidateFamily(stored.getFamily());
        throw new TokenReuseException("Token reuse detected — please login again");
    }

    // Kiểm tra hết hạn
    if (stored.isExpired()) {
        throw new AuthException("Refresh token expired");
    }

    // Cập nhật atomic, bắt trường hợp hai request dùng cùng token
    // UPDATE refresh_tokens SET used = true
    // WHERE id = ? AND used = false AND revoked = false
    int updatedRows = refreshTokenRepository.markAsUsedAtomic(stored.getId());
    if (updatedRows == 0) {
        // Request khác vừa dùng token này: coi là dùng lại,
        // vô hiệu hóa toàn bộ chuỗi token
        refreshTokenRepository.invalidateFamily(stored.getFamily());
        throw new TokenReuseException("Token reuse detected — please login again");
    }

    // Phát hành cặp token mới thuộc cùng một chuỗi (family)
    User user = stored.getUser();
    String newAccessToken = jwtUtil.generateAccessToken(user);  // 30 phút
    String newRefreshToken = createRefreshToken(user, stored.getFamily()); // 7 ngày

    return new TokenResponse(newAccessToken, newRefreshToken);
}
```

Có ba chi tiết triển khai dễ bị bỏ sót. Thứ nhất, ví dụ trên lưu token thô và nhận refresh token qua JSON body cho dễ đọc. Ở production nên lưu *hash* của refresh token (tương tự cách lưu mật khẩu) và so khớp qua hash; với client web giữ refresh token trong cookie `HttpOnly` (Bảng 9.3), endpoint đọc token bằng `@CookieValue` và trả token mới qua header `Set-Cookie`, vì JavaScript không đọc được cookie này.

Thứ hai, cặp thao tác "đánh dấu token cũ đã dùng — phát hành token mới" phải nằm trong cùng một transaction. Riêng nhánh phát hiện token bị dùng lại thì lệnh thu hồi token family phải được

commit chứ không bị rollback theo exception từ chối request — đó là lý do có `noRollbackFor` trong ví dụ; phương án khác là tách việc thu hồi vào một transaction `REQUIRES_NEW`.

Thứ ba, khi hai yêu cầu làm mới hợp lệ chạy song song, yêu cầu đến sau sẽ kích hoạt nhầm cảnh báo reuse. Hệ thống cần một chính sách rõ ràng cho tình huống này, chẳng hạn một khoảng ân hạn ngắn, để không vô hiệu hóa nhầm token vừa cấp.

Token rotation yêu cầu đặt thời gian sống (TTL) khác nhau cho từng loại token:

__Bảng 9.3:__ Access token, refresh token và token family — thời hạn, nơi lưu trữ và lý do thiết kế

Token	Thời hạn	Lưu ở đâu	Lý do
Access token	15–60 phút	Bộ nhớ client/cookie	Thời gian sống ngắn giúp giảm thiểu rủi ro nếu token bị rò rỉ
Refresh token	7–30 ngày	HttpOnly cookie + DB	Thời gian sống dài duy trì trải nghiệm liền mạch, lưu ở DB để có thể thu hồi
Token family	Theo refresh token	DB (family ID)	Giúp phát hiện việc tái sử dụng trái phép và thu hồi toàn bộ chuỗi

9.3. Tiered Trust và xác minh theo tài nguyên

Bài toán không phải chọn một trong hai cực "mọi service đều truy vấn cơ sở dữ liệu" hoặc "chỉ Gateway kiểm tra token". Cần tách ba thao tác: Gateway kiểm tra credential ở ingress; resource service xác minh bằng chứng có chữ ký dành cho mình; chỉ những luồng cần trạng thái mới truy vấn Auth Service hoặc kho thu hồi (revocation store).

Cổng trường, giấy giới thiệu và phòng chức năng

- **Gateway**: giống trạm kiểm soát cổng trường — kiểm tra credential ngoài, loại bỏ giấy tờ do người vào tự sửa và chỉ cho request hợp lệ đi tiếp.
- **Resource service**: giống phòng chức năng — kiểm tra con dấu cùng tên phòng nhận trên giấy giới thiệu, rồi tự quyết định người đó có quyền xem bảng điểm, sửa đề hay duyệt kết quả hay không. Với nghiệp vụ nhạy cảm, phòng còn kiểm tra người mang giấy đang đại diện cho ai.

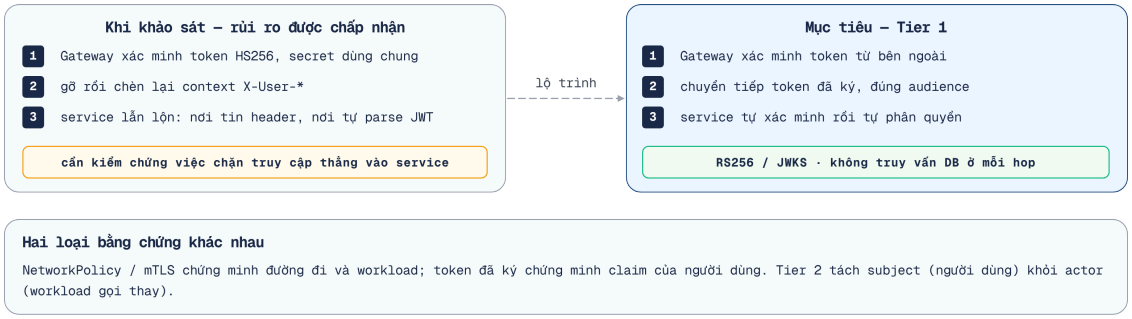
9.3.1. Vấn đề: ba phép kiểm tra thường bị gộp làm một

Xác minh mật mã (cryptographic verification) kiểm tra thuật toán, chữ ký, issuer, audience và thời hạn; thao tác này stateless và không đòi hỏi truy vấn cơ sở dữ liệu. **Kiểm tra trạng thái (state check)** hỏi tài khoản còn hoạt động hay token đã bị thu hồi; nó chỉ cần ở login, refresh hoặc thao tác nhạy cảm theo chính sách. **Phân quyền (authorization)** là quyết định nghiệp vụ trên tài nguyên cụ thể và luôn thuộc service sở hữu tài nguyên.

Nếu chỉ Gateway kiểm tra rồi thay token bằng `X-User-*`, service đích không còn bằng chứng mật mã khi có route bypass hoặc workload bị chiếm quyền. Ngược lại, nếu mỗi service tự viết verifier và tự truy vấn cơ sở dữ liệu người dùng ở mọi request, logic vừa trùng lặp vừa tạo coupling. Tiered Trust giữ bước xác minh tại resource service nhưng chuẩn hóa bước đó qua framework, issuer metadata và JWKS.

9.3.2. KBM tại thời điểm khảo sát và mục tiêu

Hình 9.6 đặt luồng định danh của KBM tại thời điểm khảo sát, với các rủi ro được chấp nhận khi đó, cạnh mục tiêu Tier 1.



Hình 9.6: *Luồng định danh KBM — rủi ro được chấp nhận tại thời điểm khảo sát và mục tiêu Tier 1*

Cả luồng tại thời điểm khảo sát lẫn mục tiêu đều được đánh giá theo bốn tầng tin cậy tóm tắt ở Bảng 9.4.

Bảng 9.4: *Bốn tầng tin cậy dùng xuyên suốt sách*

Tầng	Ranh giới	Bằng chứng	Quyết định tại service đích
T0	Client ngoài → Gateway	External token; làm sạch mọi header định danh	Gateway kiểm tra `alg`, chữ ký, `iss`, `aud`, `exp`, `nbf`
T1	Gateway/service → resource service	Signed token hoặc metadata đúng audience	Xác minh bằng chứng rồi phân quyền cục bộ
T2	Luồng nhạy cảm service-to-service	User subject + workload/actor + delegation; mTLS khi phù hợp	Giới hạn cả quyền người dùng lẫn quyền workload
T3	Producer/job → broker → consumer	Workload identity + quyền job/command + độ mới (freshness)	Consumer xác minh ngữ cảnh và phân quyền cho command

Thời điểm khảo sát — rủi ro được chấp nhận: Gateway xác minh HS256, loại bỏ rồi chèn lại header định danh; các service còn dùng lẫn lộn hai cách, tin vào header và tự phân tích JWT, nên chưa đủ căn cứ để coi mọi đường gọi nội bộ đã được kiểm soát. Vì vậy sách không gọi thiết kế này là Zero Trust hay phòng thủ chiều sâu.

Mục tiêu Tier 1: Gateway xác minh token ngoài, rồi giữ bằng chứng có chữ ký trong giai đoạn chuyển tiếp, hoặc dùng token exchange để đổi sang token nội bộ đúng audience. Mỗi resource service dùng verifier chuẩn hóa để kiểm tra bằng chứng và tự phân quyền. Auth Service chỉ kiểm tra trạng thái qua cơ sở dữ liệu khi đăng nhập, làm mới token, thu hồi token hoặc khi có thao tác nhạy cảm; không bắt buộc truy vấn cơ sở dữ liệu ở mọi hop.

⚠ Bảo mật header và truyền định danh (Identity Propagation)

Gateway bắt buộc phải loại bỏ mọi `X-User-*` do client gửi trước khi dẫn xuất context mới. Tuy nhiên, việc làm sạch header chỉ chặn spoofing tại ingress; một header văn bản thuần vẫn không phải bằng chứng danh tính. Service đích chỉ được ra quyết định dựa trên signed token/metadata đã kiểm tra issuer, audience và độ mới. NetworkPolicy giới hạn đường đi; mTLS xác thực workload; hai lớp này không thay thế chữ ký cho claim của người dùng.

Với machine-to-machine, Client Credentials đại diện cho **calling workload/actor**, không tự tạo ra user subject. Nếu service gọi thay mặt người dùng, bằng chứng phải mang subject và actor/delegation tách biệt để service đích giới hạn đồng thời quyền của cả hai. Job không có người dùng thì không được tự gắn `X-User-Id` giả để mượn quyền người dùng.

Gateway chịu trách nhiệm xác thực ở ingress nhưng không sở hữu toàn bộ quyết định tin cậy. Resource service xác minh bằng chứng tại ranh giới của mình và chịu trách nhiệm phân quyền nghiệp vụ. Sự phân công này giữ các quy tắc nghiệp vụ ngoài Gateway mà không buộc service tin mù quáng vào vị trí mạng hoặc định dạng header.

9.4. OAuth2 — nhà cung cấp danh tính bên ngoài

Ủy quyền xác thực (delegated authentication) cho các nhà cung cấp danh tính (identity provider) bên ngoài giúp giảm thiểu rủi ro bảo mật từ việc tự quản lý mật khẩu, đồng thời mang lại trải nghiệm đăng nhập một lần (SSO) liền mạch cho người dùng.

Vấn đề quản lý thông tin đăng nhập (credential): Tự quản lý thông tin đăng nhập đòi hỏi phải xử lý nhiều việc phức tạp: băm mật khẩu, cấp lại mật khẩu đã quên, xác thực đa yếu tố (2FA), chống tấn công dò mật khẩu (brute-force protection) và tuân thủ các quy chuẩn bảo mật. Với đội ngũ nhỏ, **ủy quyền xác thực** cho nhà cung cấp danh tính chuyên nghiệp (Google, Microsoft, GitHub) giúp giảm đáng kể công sức và rủi ro.

9.4.1. OAuth2 Authorization Code Flow

Đăng ký ký túc xá bằng tài khoản Cổng thông tin đào tạo

Hãy tưởng tượng Ban quản lý Ký túc xá cần xác nhận một sinh viên đang theo học trước khi xếp phòng. Ký túc xá (client) không được phép hỏi mật khẩu Cổng thông tin đào tạo của sinh viên (rất nguy hiểm). Thay vào đó, Ký túc xá chuyển hướng sinh viên sang Cổng thông tin đào tạo của trường để tự đăng nhập. Sau khi đăng nhập, trường cấp cho sinh viên một "Phiếu xác nhận" (Authorization Code) chỉ dùng được 1 lần. Hệ thống backend của Ký túc xá mang phiếu đó đến thẳng Phòng Đào tạo để đổi lấy "Giấy phép tra cứu hồ sơ" (Token) — có thời hạn và chỉ cho xem đúng những thông tin cần để xếp phòng.

Cần phân biệt hai lớp giao thức thường đi đôi với nhau: **OAuth 2.0** vốn là khung *ủy quyền truy cập* (cấp `Access Token` để gọi API thay mặt người dùng), còn **OIDC (OpenID Connect)** là lớp *xác thực danh tính* xây chồng lên OAuth 2.0, bổ sung thêm `ID Token` (một JWT có chữ ký của nhà cung cấp danh tính) để cho ứng dụng biết chính xác người dùng vừa đăng nhập là ai.

KBM tích hợp OAuth2/OIDC để thực hiện đăng nhập thông qua nhà cung cấp danh tính của trường (ví dụ: Microsoft Entra ID hoặc Google Workspace). Quy trình hoạt động như sau: người dùng được chuyển hướng tới nhà cung cấp danh tính để đăng nhập. Khi đăng nhập thành công, KBM nhận về authorization code qua callback. Tiếp đó, Auth Service sử dụng mã này để đổi lấy `ID Token` và `Access Token` thông qua giao tiếp trực tiếp giữa các máy chủ (server-to-server). Cuối cùng, dịch vụ xác minh `ID Token`, tìm kiếm hoặc tạo mới tài khoản tương ứng và phát hành JWT nội bộ của KBM. Với client không giữ được bí mật như SPA hay ứng dụng di động, **PKCE** bảo vệ authorization code: client tạo một chuỗi ngẫu nhiên, gửi bản băm của nó khi xin mã, rồi gửi chuỗi gốc khi đổi mã lấy token. Kẻ chặn được mã không có chuỗi gốc nên không đổi được token.

Điểm quan trọng: KBM *không sử dụng* trực tiếp token của bên thứ ba. Sau khi xác thực thành công, Auth Service sẽ tạo ra một **JWT nội bộ riêng của KBM** — các dịch vụ tuyến sau hoàn toàn không cần biết người dùng đã đăng nhập bằng Microsoft, Google, hệ thống Quản lý Đào tạo hay qua mật khẩu truyền thống. Cách làm này gần với ý tưởng token exchange — đổi danh tính bên ngoài lấy token nội bộ — dù không dùng giao thức RFC 8693.

9.4.2. Nhiều phương thức xác thực

KBM hỗ trợ nhiều phương thức xác thực nhưng hội tụ về cùng một token nội bộ:

Bảng 9.5: *Các phương thức xác thực trong KBM*

Phương thức	Luồng	Khi nào
Tên đăng nhập/mật khẩu	Truyền thống, hash BCrypt	Mặc định cho mọi người dùng
Microsoft/Google OAuth2/OIDC	Authorization Code Flow + PKCE (kèm `state` chống CSRF và `nonce` chống phát lại `ID Token`)	SSO bằng tài khoản trường
Cổng QLĐT (Quản lý Đào tạo)	Tích hợp tùy biến	Đăng nhập bằng tài khoản quản lý đào tạo
Xác minh email	Xác minh quyền sở hữu email trước khi kích hoạt/khôi phục	Giảm tài khoản giả, hỗ trợ cấp lại mật khẩu
Turnstile	Challenge chống bot ở biểu mẫu nhạy cảm	Bảo vệ đăng nhập, đăng ký và cấp lại mật khẩu khỏi các công cụ tự động

Tất cả các phương thức đăng nhập này đều hội tụ về một đầu ra duy nhất: **JWT nội bộ của KBM** (chứa `userId`, `roles` và `exp`). Auth Service cung cấp hàm `generateTokens(user)` — tiếp nhận đối tượng người dùng từ bất kỳ phương thức đăng nhập nào và trả về cặp `{accessToken, refreshToken}`. Các dịch vụ ở tuyến sau (downstream service) không cần phân biệt nguồn gốc xác thực. Đây là cơ sở cho cơ chế đăng nhập một lần (SSO) đa nền tảng: các module LMS core, Network Lab và DevOps Lab có thể chia sẻ danh tính nội bộ mà không buộc từng module hiểu mọi nhà cung cấp bên ngoài.

9.4.3. Service-to-service: Client Credentials Flow

Lưu ý: luồng Authorization Code dành cho tương tác có mặt người dùng (user-facing). Khi một dịch vụ chạy nền tự gọi dịch vụ khác (machine-to-machine, ví dụ Job Scheduler gọi Judge Service), nó sử dụng **Client Credentials Flow** với credential riêng để nhận access token. Token này nhận diện workload/actor và các scope được cấp cho workload; nó *không được* tự mang user subject. Nếu tác vụ thực sự thực hiện thay mặt người dùng, hệ thống cần một cơ chế delegation hoặc token exchange ghi rõ cả subject lẫn actor, đúng audience của service đích và có thời hạn ngắn.

9.4.4. Không có trình duyệt: Device Authorization Grant

Authorization Code giả định client có thể đưa người dùng qua trình duyệt và nhận callback; Client Credentials không cần trình duyệt nhưng chỉ đại diện cho ứng dụng. Một công cụ dòng lệnh cho quản trị viên, một script chạy trên máy chủ không giao diện, hay một ứng dụng trên thiết bị chỉ có bàn phím số không mở được trình duyệt, nhưng vẫn cần đăng nhập *với tư cách một người dùng cụ thể* chứ không phải với tư cách một service.

Device Authorization Grant (RFC 8628) tách đôi quá trình: thiết bị không có trình duyệt hiển thị một mã ngắn kèm một URL; người dùng mở URL đó trên *thiết bị khác* (điện thoại, máy tính), đăng nhập và nhập mã để chấp thuận; trong lúc đó thiết bị ban đầu hỏi vòng (poll) authorization server cho tới khi nhận được token. Nhờ vậy thông tin đăng nhập không bao giờ đi qua thiết bị hạn chế.

Điểm cần nhớ khi chọn luồng: câu hỏi quyết định không phải "client thuộc loại nào" mà là hai câu — *có người dùng thật đứng sau không?* và *client có mở được trình duyệt không?* Có người dùng và có

trình duyệt thì dùng Authorization Code kèm PKCE (bắt buộc với client công khai, khuyến nghị cả với client có máy chủ theo RFC 9700); có người dùng nhưng không có trình duyệt thì dùng Device Authorization Grant; không có người dùng nào cả thì đó là Client Credentials.

9.5. RBAC — Role-Based Access Control

Phân quyền giống như việc cấp thẻ từ cho các khu vực khác nhau trong trường: thẻ của sinh viên chỉ mở được cửa thư viện và giảng đường, trong khi thẻ của giảng viên có thể mở thêm được cửa phòng thí nghiệm chuyên sâu. Việc quản lý truy cập theo vai trò giúp hệ thống dễ dàng kiểm soát ai được phép làm gì.

Trong phép so sánh này, xác thực (Authentication — AuthN) là trình thẻ để chứng minh "Tôi là sinh viên Nguyễn Văn A"; phân quyền (Authorization — AuthZ) là ổ khóa điện tử kiểm tra xem thẻ của Nguyễn Văn A có vai trò `LECTURER` để mở cửa phòng nghỉ giảng viên hay không.

9.5.1. Vấn đề: ai được làm gì?

Xác thực trả lời "người dùng là ai?". Phân quyền trả lời "người dùng được làm gì?". Về nghiệp vụ, ba vai trò của KBM lồng nhau: **STUDENT** (Sinh viên) nộp bài, xem kết quả và tham gia kỳ thi; **LECTURER** (Giảng viên) cần thêm quyền tạo bài tập, khởi tạo kỳ thi và xem báo cáo thống kê; **ADMIN** thêm quyền quản lý tài khoản và cấu hình hệ thống. Tại thời điểm khảo sát, KBM chưa khai báo quan hệ kế thừa này trong Spring Security (xem phân tích bên dưới).

9.5.2. Triển khai RBAC trong KBM

KBM dùng annotation `@PreAuthorize` của Spring Security với các vai trò lưu trong JWT:

__Listing 9.2:__ Spring Security `@PreAuthorize` — RBAC dạng khai báo (declarative)

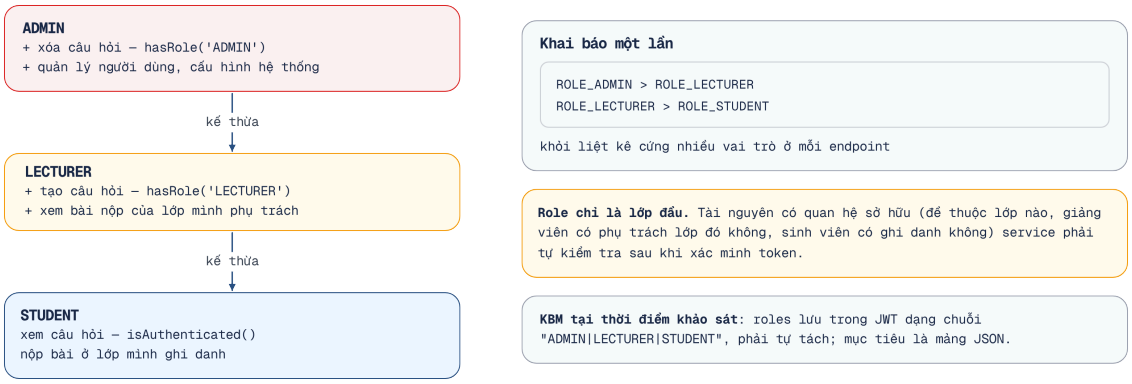
```
// Spring Security @PreAuthorize — declarative RBAC
@PreAuthorize("isAuthenticated()")
@GetMapping("/questions")
public List<QuestionResponse> getQuestions() { ... }  // Mọi user authenticated

@PreAuthorize("hasAnyRole('LECTURER', 'ADMIN')")
@PostMapping("/questions")
public QuestionResponse create(@RequestBody CreateQuestionRequest r) { ... }

@PreAuthorize("hasRole('ADMIN')")
@DeleteMapping("/questions/{id}")
public void delete(@PathVariable UUID id) { ... }
```

Kiểm tra vai trò (role check) chỉ là lớp đầu của phân quyền cục bộ. Với tài nguyên có quan hệ sở hữu (ownership) — chẳng hạn đề thi thuộc lớp nào, giảng viên có phụ trách lớp đó không, sinh viên có ghi danh hay không — service phải kiểm tra quan hệ với chính tài nguyên đó sau khi đã xác minh bằng chứng danh tính có chữ ký. Gateway không có đủ ngữ cảnh miền nghiệp vụ để quyết định thay service.

Phân cấp vai trò (role hierarchy): Sơ đồ dưới đây minh họa sự kế thừa quyền hạn giữa các vai trò:



Hình 9.7: *Phân cấp vai trò — ADMIN kế thừa LECTURER, LECTURER kế thừa STUDENT*

Để cấu hình `RoleHierarchy` cho `@PreAuthorize` và tránh phải kiểm tra cứng nhiều vai trò, chúng ta khai báo một `@Bean` (Listing 9.3). Method security chỉ tự dùng bean này từ Spring Security 6.3 (Spring Boot 3.3) trở đi, khi bật `@EnableMethodSecurity`:

Listing 9.3: *Cấu hình phân cấp vai trò `RoleHierarchy` trong Spring Security*

```
@Bean
public RoleHierarchy roleHierarchy() {
    return RoleHierarchyImpl.fromHierarchy(
            "ROLE_ADMIN > ROLE_LECTURER\nROLE_LECTURER > ROLE_STUDENT");
}
```

Tại thời điểm khảo sát, KBM lưu danh sách vai trò dạng **pipe-delimited** trong JWT: `"ADMIN|LECTURER|STUDENT"`; Gateway còn dẫn xuất `X-User-Roles` và Core Service phân tích chuỗi này. Ở mô hình mục tiêu, service lấy vai trò/quyền hạn từ token hoặc metadata có chữ ký đã được xác minh; header dẫn xuất chỉ là context, không phải nguồn quyền độc lập.

9.5.3. Điều khiển hiển thị route từ API quyền (frontend)

Frontend của KBM sử dụng một pattern bổ trợ: **danh sách quyền hiển thị route được truy xuất động từ API** (`GET /api/auth/me/permissions`) thay vì gắn cố định — response chứa danh sách route và thao tác mà người dùng được phép thấy trên giao diện. Ưu điểm của cách làm này là khi điều chỉnh phân quyền ở phía máy chủ (backend), thanh điều hướng và giao diện máy khách tự động thích ứng mà không cần triển khai lại frontend. Tuy nhiên, việc ẩn/hiện route trên giao diện chỉ phục vụ trải nghiệm người dùng (UX), hoàn toàn *không phải* rào chắn bảo mật: người dùng vẫn có thể gọi thẳng HTTP API, nên mọi kiểm tra phân quyền bắt buộc phải được thực thi độc lập tại backend.

RBAC trong KBM

Cách triển khai của KBM tại thời điểm khảo sát có một số hạn chế: (1) Các vai trò (role) được lưu dưới dạng chuỗi phân tách bằng dấu gạch đứng (pipe-delimited string) thay vì cấu trúc mảng. Điều này buộc hệ thống phải tự phân tích chuỗi, dễ phát sinh lỗi nếu tên vai trò có chứa ký tự phân cách. (2) Các annotation `@PreAuthorize` nằm rải rác ở từng controller — gây khó khăn cho quá trình kiểm toán (audit) quyền hạn của một vai trò cụ thể trên toàn bộ các endpoint. (3) KBM thiếu một hệ thống phân cấp vai trò (role hierarchy) rõ ràng ở tầng Spring Security — dẫn đến

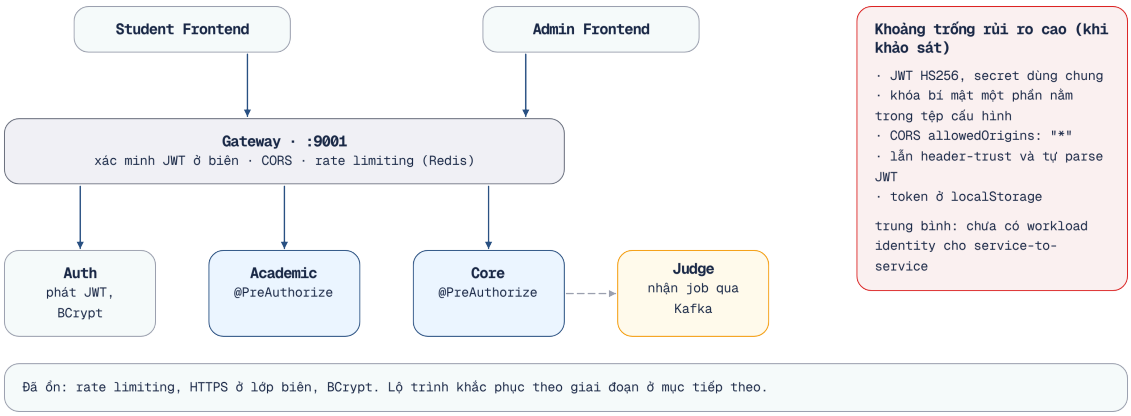
việc vai trò ADMIN phải liệt kê lại toàn bộ các quyền của LECTURER và STUDENT. (4) Phân hệ DevOps Lab cần bổ sung các chính sách kiểm soát tại tầng thực thi (runtime layer) để xác định rõ người dùng nào được phép khởi tạo không gian làm việc, vùng mạng cách ly, container hoặc các công việc đòi hỏi đặc quyền cao.

Lộ trình chuyển đổi: (1) chuyển cách lưu vai trò sang mảng JSON bên trong JWT payload, (2) thiết lập cấu hình `RoleHierarchy` thống nhất trong Spring Security, và (3) cân nhắc gom quy tắc phân quyền cấp endpoint về một chỗ (`authorizeHttpRequests` hoặc một `AuthorizationManager` dùng chung trong Spring Security), hoặc tách sang một công cụ chính sách chuyên dụng như Open Policy Agent khi số quy tắc lớn.

9.6. Case study: KBM

Phần này tổng hợp đánh giá kiến trúc bảo mật của KBM tại thời điểm khảo sát (đầu năm 2026), nhận diện các điểm yếu và đề xuất lộ trình khắc phục theo từng giai đoạn. Các điểm nêu ra mô tả mốc đó để rút bài học, không dùng để suy ra cấu hình đang vận hành.

Hình 9.8 tóm tắt kiến trúc bảo mật tổng thể của KBM.



Hình 9.8: *Kiến trúc bảo mật tổng thể của KBM*

9.6.1. Phân tích tổng hợp

Bảng dưới đây tổng hợp các khía cạnh bảo mật, rủi ro tương ứng và đề xuất biện pháp phòng ngừa:

Bảng 9.6: *Phân tích bảo mật KBM*

Khía cạnh	Tại thời điểm khảo sát	Mức rủi ro	Giải pháp khuyến nghị
Thuật toán JWT	HS256 (đối xứng, shared secret)	● Cao	RS256 + JWKS; chỉ issuer giữ private key
Lưu trữ token	Client-side (`localStorage`)	● Cao	HttpOnly cookie hạn chế JavaScript lấy token; vẫn phải phòng XSS và CSRF
Secret Management	Một phần gắn cố định trong tệp cấu hình	● Cao	Vault, AWS Secrets Manager
CORS	`allowedOrigins: "*"`	● Cao	Chỉ cho phép các origin cụ thể
Truyền định danh	Lẫn header-trust và tự phân tích JWT rời rạc	● Cao	Bằng chứng có chữ ký đúng audience + phân quyền cục bộ

Khía cạnh	Tại thời điểm khảo sát	Mức rủi ro	Giải pháp khuyến nghị
Xác thực service-to-service	Chưa có workload identity thống nhất	● Trung bình	Workload token/mTLS; delegation khi có user subject
Rate Limiting	Đã có: token bucket (Redis) ở Gateway (Chương 8)	● Thấp	Chính sách riêng cho kỳ thi
Xác minh email / Chống bot	Có/đang hoàn thiện theo module	● Trung bình	Xác minh email + Turnstile cho biểu mẫu nhạy cảm
Cô lập DevOps Lab	Runtime isolation riêng	● Cao nếu cấu hình sai	Sysbox/k3s + NetworkPolicy/RBAC
Kiểm tra tính hợp lệ dữ liệu đầu vào	Một phần	● Trung bình	Sử dụng Bean Validation (`@Valid`)
HTTPS	Kết thúc TLS ở lớp biên (reverse proxy hoặc Gateway)	● Thấp	Ghi rõ điểm kết thúc TLS trong tài liệu triển khai; cân nhắc mTLS nội bộ khi rủi ro tăng
Băm mật khẩu	BCrypt	● Thấp	—

Với HS256 mà KBM dùng tại thời điểm khảo sát, ràng buộc về độ dài khóa thường bị bỏ qua.

⚠ Độ dài khóa HS256

Với thuật toán HS256, khóa bí mật bắt buộc phải là một chuỗi ngẫu nhiên có độ dài tối thiểu 256 bit (32 byte). Các thư viện JWT hiện đại (như `jjwt` bản mới) sẽ ném ra `WeakKeyException` lúc khởi động nếu khóa quá ngắn.

9.6.2. Lộ trình cải tiến

Giai đoạn 1 — Các cải tiến nhanh (Quick Wins) (nỗ lực triển khai thấp, hiệu quả cao):

- Giới hạn origin của CORS (lưu ý: nếu kích hoạt HttpOnly cookie kéo theo `allowCredentials=true`, hệ thống tuyệt đối không được sử dụng `allowedOrigins("*")` — đặc tả CORS buộc trình duyệt chặn tổ hợp wildcard + credential, đây là quy tắc bảo mật, không phải vấn đề tương thích).
- Di chuyển secret ra khỏi tệp cấu hình sang biến môi trường (mức tối thiểu).
- Lưu trữ token trong HttpOnly cookie thay vì `localStorage`.
- Kích hoạt xác minh email và Turnstile cho đăng nhập, đăng ký, cấp lại mật khẩu ở các luồng giao tiếp công khai.
- Gỡ bỏ mọi `X-User-*` từ ingress, giữ bằng chứng có chữ ký cho dịch vụ tuyến sau và xác minh cấu hình chặn truy cập trực tiếp tới service.
- Chuẩn hóa ngay hợp đồng xác minh: thuật toán được phép (`alg`), issuer, audience, `exp`, `nbf`; không để mỗi service tự chọn quy tắc.

Giai đoạn 2 — Tăng cường xác thực (Authentication Hardening) (nỗ lực triển khai trung bình):

- Chuyển đổi thuật toán ký từ HS256 sang RS256 và công bố khóa công khai qua JWKS.
- Dùng Spring Security OAuth2 Resource Server hoặc middleware tương đương tại Gateway và các resource service.
- Triển khai luồng làm mới token kết hợp luân phiên refresh token (token rotation — mỗi lần làm mới sẽ vô hiệu hóa refresh token cũ).
- Chuẩn hóa SSO theo mô hình đổi token nội bộ có issuer/audience rõ ràng và hợp đồng claim chung.

Giai đoạn 3 — Workload Identity và Delegation (nỗ lực triển khai cao, áp dụng khi mở rộng quy mô):

- Xác thực service-to-service bằng workload token và/hoặc mTLS.
- Tách subject khỏi actor; chỉ cho phép on-behalf-of khi có delegation rõ ràng, đúng audience và thời hạn ngắn.
- Với job bất đồng bộ, ký hoặc ràng buộc command context để consumer xác minh quyền thực thi (Tier 3).
- Quản lý secret tập trung (HashiCorp Vault).
- Chính sách phân quyền ở mức API (Open Policy Agent).
- Rào chắn runtime cho DevOps Lab: Kubernetes RBAC, NetworkPolicy, resource quota và Sysbox isolation.

9.6.3. Secret Management trong production

Bảng 9.6 xếp Secret Management ở mức rủi ro cao. Đặt khóa bí mật trong tệp cấu hình đi cùng mã nguồn là lỗi phổ biến trong các dự án microservices và cần được khắc phục ở mọi môi trường production.

Bảng 9.7 so sánh ba cấp độ quản lý secret, từ biến môi trường đến giải pháp cho quy mô doanh nghiệp.

Bảng 9.7: *Chiến lược Secret Management — từ cơ bản đến quy mô doanh nghiệp*

Cấp độ	Cách tiếp cận	Ưu điểm	Nhược điểm	Phù hợp
Cấp 1	Biến môi trường	Đơn giản, tách secret khỏi code	Secret ở dạng văn bản thuần trên server, không có nhật ký kiểm toán (audit trail)	Dev/Staging
Cấp 2	Cấu hình mã hóa (Encrypted Config: `{cipher}` của Spring Cloud Config, Mozilla SOPS hoặc Jasypt)	Secret mã hóa trong repo, giải mã lúc runtime	Vẫn cần quản lý khóa mã hóa	Production quy mô nhỏ
Cấp 3	Secret Manager (HashiCorp Vault, AWS Secrets Manager, GCP Secret Manager)	Tự động luân phiên, nhật ký kiểm toán, chính sách truy cập, secret động (dynamic secret)	Chi phí hạ tầng, độ phức tạp vận hành	Production quy mô lớn

Cấp 1 — Biến môi trường (mức tối thiểu, nên áp dụng ngay):

Thay vì:

```
# SAI: application.yml — KHÔNG BAO GIỜ commit thông tin nhạy cảm vào Git
jwt:
  secretKey: "gia-tri-bi-mat-viet-truc-tiep"
  expiration: 86400000
```

Sử dụng:

```
# ĐÚNG: application.yml — tham chiếu biến môi trường
jwt:
  secretKey: ${JWT_SECRET_KEY}
  expiration: ${JWT_EXPIRATION:1800000}   # 30 phút
```

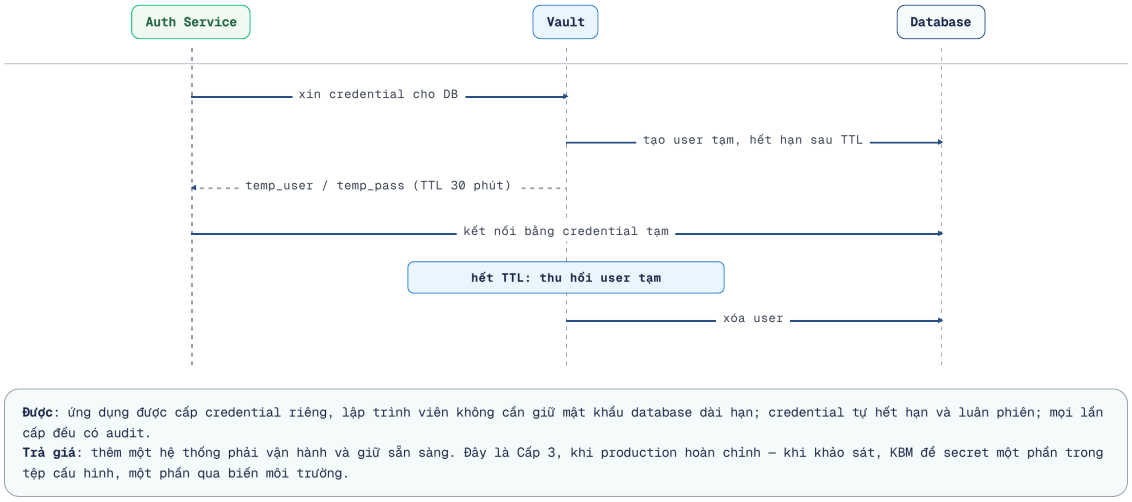
```
# Docker Compose — nạp từ file .env (không commit .env)
services:
  kbm-auth:
    environment:
```

```
      - JWT_SECRET_KEY=${JWT_SECRET_KEY}
      - DB_PASSWORD=${DB_PASSWORD}
```

Bước đầu tiên và quan trọng nhất là thêm `.env` vào `.gitignore`, đồng thời tạo `.env.example` không chứa giá trị thực để các thành viên trong nhóm phát triển biết những biến cần thiết lập. Đây là cải tiến bảo mật không tốn chi phí mà mọi dự án đều nên áp dụng ngay.

Cấp 3 — HashiCorp Vault (khi hệ thống production hoàn chỉnh):

Vault cung cấp **secret động (dynamic secret)** — thông tin đăng nhập cơ sở dữ liệu được tạo tự động, có thời hạn và tự động luân phiên. Ứng dụng được cấp credential riêng; lập trình viên không cần giữ mật khẩu database dài hạn. Khả năng đọc credential vẫn phụ thuộc chính sách truy cập Vault.



Hình 9.9: *Vault Dynamic Secret — thông tin đăng nhập tạm thời, tự động luân phiên*

⚠ Các lỗ hổng và sai lầm bảo mật thường gặp

- **Lưu JWT trong localStorage**: Bất kỳ đoạn mã JavaScript nào chạy trên trang cũng đều có thể đọc được dữ liệu này (tấn công XSS). Hậu quả: nếu tồn tại lỗ hổng XSS, kẻ tấn công có thể đánh cắp token và mạo danh người dùng. *Phòng tránh*: lưu trữ JWT trong HttpOnly cookie (JavaScript không đọc được), kết hợp `Secure`, `SameSite=Lax/Strict` và CSRF token hoặc kiểm tra Origin cho các yêu cầu thay đổi trạng thái — vì trình duyệt tự động đính kèm cookie, HttpOnly chỉ bảo vệ tính bí mật của token chứ không tự ngăn được tấn công CSRF.
- **Gắn cố định secret trong mã nguồn**: Đặt khóa bí mật trực tiếp trong `application.yml` và đưa lên Git. Hậu quả: ai có quyền truy cập repo đều có thể tạo JWT giả mạo. *Phòng tránh*: dùng biến môi trường (tối thiểu), hoặc trình quản lý secret chuyên dụng (Vault, AWS Secrets Manager).
- **Không đặt thời hạn hoặc đặt thời hạn access token quá dài**: Access token không có `exp` hoặc có thời hạn quá dài mà không đi kèm cơ chế làm mới hay luân phiên. Hậu quả: nếu token bị lộ, kẻ tấn công có thể sử dụng trong thời gian dài (thậm chí vĩnh viễn) mà hệ thống rất khó thu hồi. *Phòng tránh*: dùng access token ngắn hạn (15–60 phút), kết hợp refresh

token có thời hạn dài hơn (7–30 ngày) đi kèm cơ chế luân phiên refresh token và giám sát tái sử dụng trái phép để vô hiệu hóa toàn bộ chuỗi token.

- **Dùng HS256 shared secret cho toàn bộ môi trường production**: Tất cả các dịch vụ chia sẻ chung một khóa bí mật để ký và xác minh JWT. Hậu quả: nếu chỉ một dịch vụ bị xâm nhập, kẻ tấn công có thể tạo token giả mạo cho bất kỳ người dùng nào trên toàn hệ thống. *Phòng tránh*: chuyển sang thuật toán bất đối xứng RS256/JWKS — chỉ Auth Service giữ khóa riêng tư để ký, còn các dịch vụ khác chỉ dùng khóa công khai để xác minh.
- **Xác minh JWT ở mỗi dịch vụ theo cách khác nhau**: Mỗi dịch vụ sử dụng một phiên bản thư viện JWT và tập quy tắc riêng. Hậu quả không chỉ là Gateway chấp nhận nhưng service từ chối; một bộ xác minh lỏng hơn còn có thể chấp nhận sai thuật toán, issuer hoặc audience. *Phòng tránh*: chuẩn hóa bước xác minh tại resource service bằng cùng framework, issuer metadata/JWKS và kiểm thử hợp đồng (contract test). Gateway kiểm tra ở ingress; service đích vẫn xác minh bằng chứng có chữ ký và thực hiện phân quyền cục bộ (§9.3).
- **Tin tưởng dữ liệu định danh thô do client gửi**: Đọc thẳng header như `X-User-Id` hoặc tin claim chỉ vì request đi qua mạng nội bộ. Hậu quả: leo thang đặc quyền và tấn công confused deputy. *Phòng tránh*: Gateway gỡ bỏ các header định danh từ client; bằng chứng có chữ ký phải được service đích kiểm tra chữ ký, issuer, audience và thời hạn trước khi phân quyền. Header dẫn xuất không được trở thành nguồn quyền độc lập.
- **Confused Deputy Problem**: Dịch vụ A gọi dịch vụ B thay mặt người dùng, nhưng B không biết A đang "đại diện" cho người dùng hay hành động với đặc quyền riêng của A. Newman trong [4a, Ch.9] gọi đây là *confused deputy problem*. Hậu quả: người dùng chỉ có quyền `STUDENT` nhưng A lại gọi B bằng credential hệ thống toàn quyền. *Phòng tránh*: bằng chứng thay mặt người dùng (on-behalf-of proof) phải tách danh tính người dùng (*subject*) khỏi định danh tác nhân gọi (*actor/workload*), ghi rõ phạm vi ủy quyền (delegation), audience và thời hạn. B xác minh bằng chứng rồi giới hạn quyền theo cả subject lẫn actor; header văn bản thuần `X-User-*` không đủ để biểu diễn quan hệ này.

9.6.4. Cô lập thực thi và phong tỏa (lockdown) thi cử — hai bài học phòng thủ

Bảo mật không dừng ở JWT và RBAC. Hai nghiệp vụ chấm code và thi phòng máy của KBM cho thấy hai mặt khác của phòng thủ: **cô lập tài nguyên** khi phải chạy mã không tin cậy, và **phòng thủ chiều sâu** khi chống gian lận.

Chạy mã sinh viên: rủi ro không có sandbox và lời giải cô lập

Bộ chấm code thế hệ đầu (`kbm-judge-code` Gen 1) chạy bài nộp C/C++ và Java **trực tiếp trong tiến trình máy chủ, không có sandbox**. Đây là lỗ hổng nghiêm trọng: mã sinh viên là *dữ liệu đầu vào không đáng tin cậy có khả năng thực thi* — nó có thể đọc/ghi file hệ thống, thiết lập kết nối mạng trái phép ra bên ngoài, tiêu thụ vô hạn CPU/RAM, hoặc cố ý gây ngưng trệ dịch vụ. Khác với chấm SQL (chạy trong sandbox cơ sở dữ liệu cô lập mỗi lần nộp bài) hay DevOps Lab (pod Sysbox cô lập), Code Judge Gen 1 bỏ qua hoàn toàn lớp cô lập — một dạng đánh đổi "tự viết để kiểm soát tối đa" nhưng vô tình mở rộng bề mặt tấn công ngay bên trong ranh giới tin cậy của dịch vụ.

⚠ Không bao giờ chạy mã không tin cậy (untrusted code) chung tiến trình

Mã do người dùng nộp lên (bài nộp cho bộ chấm, plugin, công thức tùy biến...) phải được xem như **thù địch cho đến khi được chứng minh ngược lại**. Chạy nó chung tiến trình/máy chủ với

service đồng nghĩa trao cho kẻ tấn công chính quyền hạn của service đó: truy cập file, mạng, secret trong biến môi trường, và tài nguyên không giới hạn. Lỗ hổng này không phải lỗi logic mà là *thiếu một lớp cô lập*.

Thế hệ hiện tại tích hợp **DOMjudge**, chạy mỗi bài nộp trong **sandbox cô lập** dựa trên cgroup/namespace của Linux: giới hạn CPU, bộ nhớ, thời gian thực thi, và cắt quyền truy cập tài nguyên ngoài. Nguyên tắc bảo mật cốt lõi: **không bao giờ chạy mã không tin cậy trong cùng ranh giới tin cậy với dịch vụ**. Ở đây, sandbox đi song hành với quyết định xây hay mua (build vs buy) (Chương 3, Chương 4). Việc lựa chọn một hệ thống chấm điểm mã nguồn mở (OSS judge) đã được kiểm chứng qua nhiều kỳ thi ICPC vừa giải quyết bài toán đa ngôn ngữ, vừa khắc phục lỗ hổng thiếu cô lập mà phiên bản tự phát triển trước đó gặp phải.

Phong tỏa thi cử và phòng thủ chiều sâu chống gian lận

Với kỳ thi tại phòng máy, KBM dùng `kbm-exam-client` — một desktop client kiểm soát chặt môi trường thi: bắt buộc toàn màn hình, chặn Alt-Tab/chuyển ứng dụng, vô hiệu hóa copy-paste và chuột phải, chặn mở trình duyệt/tài liệu ngoài, và chỉ cho kết nối từ **danh sách IP được phép (IP whitelist)** của phòng thi. Client còn thu tín hiệu giám sát (rời cửa sổ, mất focus, cắm/rút thiết bị) và đẩy về server làm dữ liệu chống gian lận (anti-cheat). Client này dùng cùng JWT của `kbm-auth` (SSO) và là một client của Bounded Context Examination & Proctoring.

Tuy nhiên, việc phong tỏa (lockdown) ở phía máy khách **không bao giờ được coi là lớp phòng thủ duy nhất**. Một thí sinh có kỹ thuật có thể vá, giả lập hoặc bỏ qua client desktop — mọi thứ chạy trên máy người dùng đều nằm dưới quyền kiểm soát của người dùng. Vì vậy server (logic kỳ thi và chống gian lận, hiện trong `kbm-core`) phải **độc lập** phát hiện bất thường: đổi IP hoặc User-Agent giữa ca thi, tần suất nộp bài bất thường, số lần vi phạm vượt ngưỡng cấu hình. Đây chính là **phòng thủ chiều sâu** (§9.1) áp dụng cho anti-cheat: tín hiệu được thu cả ở client (lockdown) lẫn ở server (phát hiện bất thường), không phía nào là điểm tin cậy tuyệt đối.

Lockdown của `kbm-exam-client` chỉ làm *tăng chi phí gian lận*, không *loại bỏ* khả năng gian lận. Cũng theo nguyên lý Gateway gỡ bỏ header `X-User-*` từ Internet (§9.3), mọi tín hiệu bảo mật do client gửi lên đều phải được server kiểm chứng hoặc đối chiếu độc lập. Client đảm nhiệm trải nghiệm người dùng và là lớp ngăn chặn đầu tiên; server nắm quyền quyết định cuối cùng và là căn cứ tin cậy duy nhất cho anti-cheat.

Các pattern trong chương có minh họa tương tác đi kèm tại companion web:

Interactive Demo

- **Luồng xác thực OAuth2 & JWT**: oauth2-jwt-flow.html
- **Role-Based Access Control (RBAC)**: rbac-authorization.html

9.7. Tổng kết

Bảo mật microservices phức tạp hơn bảo mật hệ thống nguyên khối (monolith) vì bề mặt tấn công lớn hơn — mỗi dịch vụ, mỗi lời gọi mạng, mỗi cơ sở dữ liệu đều là điểm tiềm ẩn rủi ro. Phòng thủ chiều sâu (defense in depth) — bảo vệ nhiều lớp thay vì dựa vào một lớp duy nhất — là nguyên tắc nền tảng.

JWT cung cấp các claim có chữ ký số theo cách stateless, không cần kho lưu trữ phiên chung cho mỗi request. HS256 (ký bằng khóa đối xứng) đơn giản nhưng mọi bên xác minh giữ shared secret cũng có khả năng tự ký token — chỉ nên dùng trong phạm vi hẹp và được kiểm soát chặt. RS256 (ký bằng cặp khóa bất đối xứng) tách bên ký khỏi bên xác minh: Auth Service giữ khóa riêng tư (private key), còn các resource service lấy khóa công khai (public key) qua JWKS. Dù dùng thuật toán nào, bên xác minh vẫn phải cố định thuật toán được phép, đối chiếu issuer và audience, rồi kiểm tra thời hạn token.

Mô hình tin cậy phân tầng (Tiered Trust) tách ba lớp kiểm tra: kiểm tra ở biên (Gateway xác minh token tại ingress), xác minh tại dịch vụ đích (service đích kiểm tra chữ ký và quyền truy cập) và đối chiếu trạng thái tại cơ sở dữ liệu. Tier 1 yêu cầu bằng chứng có chữ ký cùng phân quyền cục bộ tại service sở hữu nghiệp vụ; Tier 2 bổ sung workload identity, actor và ủy quyền (delegation); Tier 3 bảo vệ command bất đồng bộ. NetworkPolicy và mTLS thu hẹp đường đi mạng, nhưng không thay thế bằng chứng cho danh tính người dùng (user subject). OAuth 2.0 và OIDC cho phép giao việc xác thực người dùng cho nhà cung cấp danh tính chuyên dụng; Client Credentials chỉ đại diện cho workload (dịch vụ gọi dịch vụ) nếu không có delegation rõ ràng.

RBAC (phân quyền theo vai trò) là điểm khởi đầu phổ biến cho bài toán phân quyền. KBM triển khai RBAC qua `@PreAuthorize`, nhưng kiểm tra vai trò phải đi cùng kiểm tra quyền sở hữu tài nguyên (ownership) và quan hệ nghiệp vụ tại service đích — ví dụ, sinh viên chỉ xem được bài nộp của chính mình dù cùng vai trò `STUDENT`. Tại thời điểm khảo sát, hệ thống còn thiếu phân cấp vai trò (role hierarchy) rõ ràng và một cách khai báo chính sách phân quyền dễ kiểm toán.

Phân tích KBM tại thời điểm khảo sát cho thấy hệ thống đã có JWT, Gateway, RBAC và SSO, nhưng truyền định danh còn là rủi ro được chấp nhận có kiểm soát. Lộ trình mục tiêu là làm sạch request ở biên, thống nhất hợp đồng xác minh, chuyển sang RS256/JWKS cùng bằng chứng danh tính có chữ ký cho chặng nội bộ, rồi bổ sung workload identity/delegation theo tầng tin cậy. Các khoản nợ kỹ thuật khác — CORS, quản lý secret, lưu trữ token và cô lập thực thi — vẫn phải được xử lý song song.

Ở Chương 10, chúng ta sẽ vận dụng kiến thức từ Chương 1–9 vào **một bài toán thực tế**: chuyển đổi từ monolith sang microservices — khi nào nên chuyển, Strangler Fig pattern, tách database, và lộ trình chuyển đổi cho KBM.

9.8. Đọc thêm

Sách tham khảo chính:

1. [4a] Sam Newman, *Building Microservices* — Ch.9: Security
2. [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. — Ch.11: Developing production-ready services (§11.1: Developing secure services, Access Token pattern)
3. [7] Martin Kleppmann, *Designing Data-Intensive Applications* — Ch.4: Encoding and Evolution (data formats, schema evolution — nền tảng để hiểu binary/JSON encoding trong token và thông điệp API)

Nguồn trực tuyến:

- JWT.io — jwt.io (interactive JWT decoder)
- OWASP API Security Top 10 — owasp.org/www-project-api-security
- Spring Security OAuth2 Resource Server — docs.spring.io/spring-security
- RFC 7519 — JSON Web Token (JWT) specification
- NIST SP 800-207 — Zero Trust Architecture

- RFC 8725 — JSON Web Token Best Current Practices
- RFC 9068 — JWT Profile for OAuth 2.0 Access Tokens
- RFC 9700 — Best Current Practice for OAuth 2.0 Security

Bài tập: 9-0, 9-1, 9-2 ở phần Bài tập & case study cuối sách.

Tài liệu tham khảo

[2a] Richardson, C. (2018). *Microservices Patterns: With Examples in Java*. Manning Publications.

[4a] Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media.

[7] Kleppmann, M. (2017). *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media.